

AI 落地实践：从个人工具到企业交付

40 分钟逐字稿 V1。语气按现场平实口播处理，保留少量自然互动。

P01 · 从个人工具到企业交付

大家好，我是阿亮。

我自己做研发大概十二年，这两年主要在做 AI Agent、AI 工程化，还有一些从个人工具到企业项目的落地。

今天这场分享，我想尽量讲得具体一点，不讲太多概念堆叠，也不做工具参数对比。因为如果只讲工具，大家很容易听完以后觉得：好像很热闹，但回去还是不知道第一步怎么做。

我今天想回答的是一个更实际的问题：一个普通人，或者一个小团队，怎么真的用 AI 做出一个产品？不是写一个 demo，不是跑一个 prompt，而是把一个东西做出来，让它能用，能迭代，甚至能交付给真实客户。

这份 PPT 本身，也是用我现在这套工作方式做出来的。我写 brief，整理资料，让 AI 生成页面和讲稿，然后我来检查、调整、删掉不合适的表达。

所以今天不是讲一个远处的趋势，而是讲我现在正在用的工作方法。

主线很简单：从个人工具，到企业交付。我们从一个人做第一个小产品开始，讲到怎么做第 N 个，再讲到这套方法进企业以后会遇到什么问题。

P02 · 你需要的不是更快的程序员，而是更清晰的判断

我想先从一个很常见的体验说起。

大家可以回想一下，你有没有让 AI 写过代码，或者写过方案、文案、页面。它写得很快，一秒钟刷出来很多内容，看起来也挺完整。

但你再看一会儿，会发现一个问题：它好像很努力，可是它并不一定知道你真正想要什么。

代码能跑，不代表产品是对的。页面好看，不代表用户会用。功能很多，也不代表问题被解决了。

我自己刚开始用 AI 写代码的时候，也会被速度吸引。它能很快生成一堆文件，改一堆逻辑，看起来像一个特别勤快的程序员。但后来我慢慢发现，速度不是最大的瓶颈。

真正的瓶颈，是判断。

判断什么？判断这个需求值不值得做，范围该切到哪里，哪些东西先不做，哪些地方必须人工检查。判断一个 bug 是真的影响用户，还是只是自己看着不舒服。判断 AI 给出的方案，是不是在绕远路。

AI 会让你快很多，但前提是方向对。方向错的时候，它也会让你错得更快。

所以今天第一句话，我希望大家先记住：你需要的不是一个更快的程序员，你需要的是更清晰的判断。

如果你的判断清楚，AI 是放大器。如果你的判断不清楚，AI 就会把混乱也一起放大。

P03 · 为什么大多数人卡在第 0 个产品

很多人不是做不出产品，而是根本没开始。

我身边有不少这样的朋友。他们有想法，而且不是没价值的想法。有的人想做一个效率工具，有的人想做一个小程序，有的人想做一个课程辅助系统。聊起来都很清楚，但两个月过去，半年过去，还是停在“我再想想”。

这里面通常有三个卡点。

第一个，是等一个完美的想法。总觉得这个点还不够大，那个功能还不够完整，名字还没想好，商业模式还没想透。但第一个产品真的不需要这么重。第一个产品最重要的不是完美，而是存在。它要从脑子里出来，变成一个能被打开、能被使用的东西。

第二个，是把“不会编程”当成门槛。过去这确实是门槛，但现在变了。Vibe Coding 的核心，不是你会不会写语法，而是你能不能说清楚意图。你不需要一开始就会写 for 循环，你需要能说清楚：谁来用，解决什么问题，最小版本长什么样。

第三个，是一上来就想做平台。这个特别常见。明明只是想解决一个小问题，最后越想越大，变成账号系统、后台管理、数据分析、权限、支付、社区。越想越完整，也越不可能开始。

这里我想和大家做个小互动。你可以在心里想一下，你最近有没有一个拖了很久的产品想法？它没有开始，真正的原因是技术不够，还是范围太大？

多数时候，卡住我们的不是技术，是我们默认了一句：我还没准备好。

但做第一个产品，准备好不是前提。做出来，才会让你准备好。

P04 · 把意图变成产品的最短路径

那怎么开始？

我自己的经验是，把整个过程拆成三段：意图、规格、执行。

意图，就是你用大白话说清楚你要什么。这个阶段不用追求专业，也不用写得像产品文档。你可以很粗糙地说：我每天都要重复做这件事，很烦，我希望有个工具自动帮我做。它给谁用，解决什么问题，最小版本做到哪一步。先说出来。

规格，是把这个意图变成 AI 能稳定执行的文档。这里会更结构化一点：功能范围、用户流程、验收标准、技术限制、边界条件。规格不是为了显得专业，而是为了减少误解。

这里有个点很重要：规格里最值钱的部分，往往不是“做什么”，而是“不做什么”。

比如你说“优化一下体验”，这句话对 AI 来说太宽了。它不知道你是要改按钮、改动效、改性能，还是改页面结构。但如果你说“点击按钮以后 300 毫秒内要出现反馈；这次不做登录、不做多语言、不做后台配置”，AI 就会稳很多。

执行，就是 AI 按规格去写代码、跑测试、修问题。这个阶段你不要盯着每一行代码看。你要看关键节点：方向有没有偏，范围有没有膨胀，核心逻辑有没有错。

所以这三段里，人最关键的工作，不是打字写代码，而是把意图说清楚，把边界定清楚，在关键地方做判断。

这也是为什么我说，未来很多人的门槛不再是“会不会写代码”，而是“能不能把需求说清楚”。

P05 · 从写代码，到表达意图

这一页我想讲一个很具体的变化：过去做产品，第一步经常不是做产品，而是先跨过一堆技术门槛。

你要先学语法，学框架，配环境。好不容易跑起来了，再一行一行写实现。中间还会掉进调试黑洞，环境问题、依赖问题、低级 bug，都会消耗大量时间。

所以过去一个人的产品周期，很多时候是以月来算的。不是因为想法一定很复杂，而是启动成本太高。

但现在这件事变了。

新范式不是让你完全不思考，也不是让 AI 随便替你做决定。新范式是：你用大白话把需求讲清楚，AI 帮你拆规格、写实现、跑测试，你在关键节点做判断。

也就是说，你的主要工作从“我怎么把每一行代码写出来”，变成“我到底要什么，哪些要，哪些不要，做到什么程度算完成”。

这个变化会直接压缩第一个产品的周期。以前你可能一个月还在搭环境、补基础、查文档；现在如果需求足够小，边界足够清楚，几天内就可以做出一个能用的版本。

当然，这里有个前提：你不能只说“帮我做个好用的工具”。这句话太虚了。你要说清楚：这个工具给谁用，解决哪一步，输入是什么，输出是什么，这一版先不做什么。

所以这一页其实只讲一件事：AI 把“写代码”的门槛往后推了，但它把“表达意图”的要求往前提了。

未来能力的上限，不只是你会多少语言、熟多少框架，而是你能不能把一个模糊想法，讲成 AI 可以执行的清楚任务。

P06 · 挑对你的工具和模型

工具这件事，我不建议大家一开始陷入比较。今天谁更强，明天谁更新，变化很快。真正稳定的是选工具的逻辑。

如果你是刚开始做第一个产品，我建议先看自己习惯哪种工作方式。

一种是终端派，比如 Claude Code、Codex CLI。它们住在命令行里，可以读代码、写代码、跑测试、提交，适合比较完整的项目，也适合跨文件、跨模块的修改。你如果已经有一点工程习惯，这类工具会很顺。

另一种是编辑器派，比如 Cursor。它更直观，改动能马上看到，适合前端页面、交互调整，也适合不太习惯命令行的人。

现在还有一些桌面版工具，图形界面会更友好，不一定非要碰终端。对很多非程序员来说，这反而是更好的入口。

模型怎么选？我的建议也简单：复杂任务用强模型，快速试错用便宜一点、反应快一点的模型。不要所有事都用最贵的，也不要为了省钱在关键任务上用太弱的。

这里大家可以记一个判断：如果这个任务涉及架构、跨文件理解、长时间执行、复杂 bug，模型强一点是值得的。如果只是改几句文案、调样式、生成一个小脚本，就没必要上来就用最贵的。

工具会变，模型排名也会变。但你要问自己的问题不变：我现在是在探索想法，还是在交付结果？我是在做小改动，还是让 AI 跑一段完整任务？

选工具之前，先判断自己处在哪一步。

P07 · 第一个产品，怎么诞生

如果我们把第一个产品的过程再落细一点，大概是五步。

第一步，你来说清楚需求。不要一开始就写很复杂的 PRD。先用自然语言写：我是谁，我每天遇到什么麻烦，我希望这个工具帮我完成哪一步。最好再补一句，这个版本不做什么。

第二步，让 AI 帮你整理成规格。它可以帮你把需求拆成功能、页面、数据、验收标准。这个时候你要 review，不要它写什么你都接受。你要看它有没有理解错，有没有把范围放大。

第三步，让 AI 按规格执行。写代码，跑起来，遇到错误自己修。这里很多人会忍不住盯着每一行代码看。我理解这个感觉，就像坐在副驾驶，看别人开车，总想踩空气刹车。

但你要练习放手。第一个产品不是让你证明自己会写代码，而是让你证明自己能把事情推进到完成。

第四步，你看关键决策。比如技术选型是不是太复杂，数据是否会丢，用户流程是否跑得通。不要每行都看，但关键地方一定要看。

第五步，上线，或者至少发给真实用户用。哪怕只是发给三个朋友，也比自己在本地反复打磨有价值。真实反馈会告诉你下一步该做什么。

这里我想停一下。大家如果回去真的做第一个产品，不要从“我要做一个平台”开始。请从“我每天重复做的一件小事”开始。

第一个产品越小，越容易完成。完成一次，比想十次都重要。

P08 · 让 AI 更强的进阶能力

这一页有几个名词：MCP、Skills、子智能体。听起来会有点技术，但大家不用记名词。我只讲它们分别解决什么问题。

MCP 可以理解成给 AI 接工具。默认情况下，AI 主要是在和文字打交道。你告诉它，它回答你。但接上 MCP 以后，它可以访问更多外部系统，比如文件、数据库、浏览器、内部工具。

这意味着 AI 不只是“说”，它可以开始“做”。

比如一个客户需求来了。过去你可能要整理成文档，录进系统，再通知同事。每一步都不难，但很碎。AI 如果接上这些工具，你可以让它按照规则把这几步跑完。

Skills 可以理解成把一套固定流程存下来。你每次做同类工作，总有一些步骤是重复的。比如准备演讲、整理会议纪要、生成项目方案、检查代码。你不希望每次都重新教 AI 一遍，就可以把流程封装成 Skill。

今天这套 SpeakDeck，本质上就是把“演讲准备”这件事流程化：读 brief，解析 PPT，写逐字稿，生成音频，注入页面，自检。

子智能体解决的是并行问题。以前一个人做事，一次只能处理一条线。现在你可以让不同的 AI 分别去查资料、写代码、跑测试、整理反馈。

但我也提醒一句：这些能力不是一开始就必须用。第一个产品先别追求复杂。先把“意图、规格、执行”跑通，再慢慢把流程沉淀成工具。

P09 · clip-shot：一件事，做到极致

我讲第一个真实案例，clip-shot。

这个工具很小，小到一句话就能讲清楚：当剪贴板里有图片时，自动上传，然后生成 Markdown 链接，再写回剪贴板。

它解决的是我自己的一个很具体的痛点。我经常写文档、写课件、整理资料。每次截图以后，要保存文件、上传图床、复制链接、粘贴到文档里。一次可能就三十秒，但一天做很多次，就很烦。

我后来算了一下，一年下来，可能有好几个小时，甚至更多时间，都浪费在这种没有创造力的动作上。

所以我就想，能不能让它自动完成？

这个需求很适合做第一个产品，因为它足够小。没有复杂用户系统，没有商业模式，没有后台运营。它只处理一个动作：剪贴板图片变链接。

技术上它是 macOS 菜单栏工具，用 Swift 写。Swift 不是我熟的语言。如果放在以前，我可能会觉得这件事要先学语言、学 macOS 开发、学一堆 API。学着学着，热情就没了。

但这次不一样。我把需求写清楚，让 AI 帮我做。因为范围足够小，AI 很快就能跑起来。后面我只需要看关键行为：能不能监听剪贴板，能不能上传，链接是不是正确，失败时有没有提示。

clip-shot 给我的启发是：第一个产品不要追求大。你只要找一个自己每天都遇到的小麻烦，把它解决掉，就已经迈过了最难的一步。

产品不是从宏大愿景开始的。很多时候，是从“我真的不想再手动做这件事了”开始的。

P10 · landr: AI 求职 Agent

第二个案例是 landr，量级就完全不一样。

landr 是一个 AI 求职 Agent，目标是把找工作这件事拆成可以自动化的完整流程。比如简历理解、岗位匹配、面试准备、申请追踪，还有 Chrome 扩展去采集岗位信息。

为什么我说它是从 1 到 N 的案例？因为它已经不是一个小工具了。它里面有多个模块，有数据模型，有浏览器扩展，有不同 LLM 的适配，还有真实场景里的各种麻烦。

比如岗位列表里经常只有很短的摘要，十几个字，不够判断。那怎么办？就需要后台打开详情页，把完整 JD 抓回来，再结构化。

再比如 Boss 直聘这类平台会有风控。什么能自动做，什么不能自动做，哪里要停下来让人确认，这些都要提前写进规格。

还有一个很重要的点：landr 的 Roadmap 里明确写了不做什么。比如不做自动打招呼，不做自动投递，因为这会碰到平台规则和风控风险。也不做帮你“包装”简历这种不真实的事情。

你会发现，项目越复杂，“不做什么”越重要。

这个项目里，我并不是一行一行写代码。我做的是写 SPEC，设检查点，让 AI 分阶段执行。每个阶段可以独立 review，没问题再继续。

所以从 clip-shot 到 landr，变化不是“AI 写代码更快了”这么简单。变化是工作方法变了。

小工具靠 Vibe 就能开始。复杂产品一定要靠 SPEC 稳住边界。否则 AI 会很努力地把事情做大，但不一定做对。

P11 · 同一套方法，进企业为什么不灵了

前面讲个人项目，听起来会比较顺。但这套方法进企业以后，就没那么简单了。

个人项目有一个优势：上下文都在你脑子里。你知道为什么要做，知道历史上做过什么决定，也知道哪些东西只是临时方案。你给 AI 描述的时候，很多背景虽然没写出来，但你自己能补上。

企业里不一样。上下文散落在很多地方。

有的在旧系统里，有的在接口文档里，有的在三年前的 PRD 里，有的在某个老员工脑子里。还有一些东西根本没人写过，大家只是默认“我们一直都这样”。

AI 拿不到这些背景，就很容易跑偏。

尤其是老系统。命名可能不规范，逻辑可能耦合，注释可能过期。AI 看起来读懂了，但它不一定知道哪个逻辑是核心功能，哪个逻辑是历史兼容，哪个地方绝对不能碰。

我见过一种情况，AI 把一个遗留兼容逻辑当成无用代码改掉了。单看代码，好像变干净了；但线上业务会出问题。

所以进企业以后，问题不是 AI 能不能写代码。问题是：它有没有足够上下文，它知不知道边界，它遇到不确定的地方会不会停下来问。

这里大家可以想一下：你们自己团队里，有多少关键规则其实没有写在文档里？如果一个新人进来，他要多久才能搞懂？AI 也是一样的。

企业场景里，AI 落地的关键不是更快生成代码，而是先把上下文整理出来。

P12 · 引入 SDD，覆盖更长的流程

那企业里怎么用？我现在会把它拆成几个动作。

第一个动作，是让 AI 先问问题。不要一上来就让它写方案或写代码。你先给它基本背景，然后让它反问你：为了完成这个任务，你还缺哪些信息？

这一步很有价值。因为 AI 问出来的问题，往往正好是团队内部没有对齐的地方。我们做智能客服项目时，AI 第一轮问了七个问题，其中有几个是我们自己内部也没有讲清楚的。

第二个动作，是把回答沉淀成约束。不要只是口头解释。比如某个接口不能改，某个字段历史上有兼容逻辑，某个流程必须人工确认。这些都要写进规格。

第三个动作，是把历史资料喂进去。旧 PRD、接口文档、架构图、线上问题记录、历史决策，只要和任务相关，都尽量整理出来。企业场景里，最费时间的往往不是写代码，而是整理上下文。

第四个动作，是设置检查点。AI 跑到关键节点必须停下来。比如确认范围、确认技术方案、确认数据迁移、确认上线风险。不要让它一路自动跑到底。

第五个动作，是必要时并行跑方案。AI 的一个优势是并发。你可以让它给两个方案，或者让不同智能体从不同角度检查。一个看架构，一个看测试，一个看风险。

这套方法我会叫 SDD，或者说规格驱动开发。名字不重要，重要的是思路：企业里不能只靠一句 prompt 开干，必须先把上下文、边界、验收标准整理出来。

个人项目追求快，企业项目先追求稳。稳了以后，速度才会真的出来。

P13 · 真实项目，真实数据

这里讲两个企业项目。

第一个是智能客服 Agent。这个项目是从零开始做，所以对 AI 比较友好。没有太多历史包袱，可以从需求、架构、模块边界开始设计。

最终交付的代码，大部分是 AI 写的。我们团队做的事情，更多是定义需求、设计架构、处理边界问题、做 review。

这里有个挺有意思的感受。以前我们说开发速度，主要看工程师写得快不快。现在速度取决于前面定义得清不清楚。如果需求和规格清楚，AI 可以很快推进；如果前面没想清楚，后面就会反复返工。

这个项目里，整体开发速度大概是纯人工的三到四倍。而且因为我们要求 AI 跑测试、记录关键决策，反而比一些临时赶出来的人工代码更规范。

第二个是智能营销系统。这个就不一样，它是一个有几年历史的老系统。老系统的问题不是“从零写不写得出来”，而是“你敢不敢改”。

这个项目最重要的投入，不是让 AI 立刻写代码，而是花时间整理上下文。把散落的文档、接口、业务规则、历史逻辑整理成结构化输入。这个过程大概花了一到两周。

整理完以后，后面的功能开发速度明显提升。因为 AI 不再是凭空猜，它有资料可以读，有规则可以遵守，有边界可以参考。

所以我对企业 AI 落地的判断是：AI 不是简单替代工程师，它是在放大团队的判断力和组织能力。

如果团队本来就混乱，AI 会放大混乱。如果团队能把上下文、规则和边界整理好，AI 会放大效率。

P14 · 第 N 个产品的复利

做完第一个产品以后，你会发现第二个会快很多。

不是因为你突然变成了更厉害的程序员，而是因为你有了资产。

你写过的规格，可以复用。你踩过的坑，可以变成约束。你调过的提示词，可以放进模板。你总结过的流程，可以变成 Skill。你做过的每一个产品，都在给下一个产品铺路。

这就是第 N 个产品的复利。

我自己现在会同时推进不同技术栈的项目，比如 Rust、TypeScript、Swift。换成以前，这件事对我来说压力很大。因为每个技术栈都要深入学习，很多细节都要自己查。

现在不一样。我不需要每个语言都从底层精通，才能开始做产品。我的工作重心变成：定义问题，写规格，设检查点，做 review。

这不代表技术不重要。技术仍然重要，尤其是在判断 AI 方案是否合理的时候。但技术的作用变了。它不再是进入产品创造的唯一门槛，而是你做判断时的底层能力。

这里也想提醒一句，不要因为 AI 能写代码，就完全放弃理解。你越懂基本逻辑，越能用好 AI。你不用每行都写，但你要知道什么地方危险，什么地方要停下来检查。

第一个产品最难，因为你还没有信心，也没有模板。第二个开始，你会有自己的工作流。到第三个、第四个，你会开始沉淀方法。

真正的变化，不是做出一个产品，而是你开始拥有持续做产品的能力。

P15 · 现在，轮到你了

收尾我不讲太虚的东西，只给大家两个动作。

第一个动作，本周就做一个小工具。

不要选太大的题目。不要做平台，不要做完整商业产品。就找一个你每天都烦的小动作。比如整理截图、生成周报、清理表格、分类文件、汇总资料。只做一件事，做到能用。

你可以今晚就写三句话：这个工具给谁用；解决哪个重复动作；这个版本不做什么。

第二个动作，给它写一份规格。

不用长，一页就够。写清楚做什么、不做什么、验收标准是什么。比如：输入是什么，输出是什么，什么情况算成功，哪些功能这版不做。

这份规格不是为了给别人看，而是为了让你自己想清楚，也让 AI 执行时不乱跑。

如果你愿意再往前走一步，就给这个小产品设置检查点。让 AI 在确定技术方案时停一次，在完成核心功能时停一次，在准备上线时停一次。你不需要从头盯到尾，但关键地方要拍板。

我觉得 2026 年，Agentic Coding 会真正进入落地阶段。不是因为 AI 会写代码这件事新鲜，而是越来越多人会开始理解：真正重要的能力，是把意图变成规格，再把规格交给 AI 执行。

如果今天只记住一句话，我希望是这句：

你不需要成为更快的程序员，你需要成为更清晰的判断者。

谢谢大家。